

Trabajo Práctico 2 – Meta-Learning y Domain Adaptation

I309 – Visión Artificial Avanzada
Universidad de San Andrés

Introducción

El aprendizaje supervisado clásico asume que los datos de entrenamiento y test provienen de la misma distribución. Dos de los desafíos más frecuentes en la práctica son la **escasez de datos etiquetados** y el **shift de dominio**, es decir que los datos provengan de una distribución diferente a la de entrenamiento.

Este trabajo práctico aborda ambos desafíos. En las Partes 1 y 2 se implementan algoritmos de **meta-learning** para *few-shot learning*: modelos entrenados sobre tareas de MNIST que deben generalizarse a dígitos de dominios no vistos (MNIST-M y SVHN) usando apenas unos pocos ejemplos de soporte. En la Parte 3 se implementa un método de **adaptación de dominio adversarial** que aprende representaciones invariantes al dominio para trasladar directamente el conocimiento de MNIST a MNIST-M.

A lo largo del trabajo se pedirá visualizar el espacio latente del encoder en distintas instancias del entrenamiento, como herramienta para entender qué está aprendiendo cada modelo y cómo la geometría del espacio de features impacta en la capacidad de transferencia.

Conjuntos de Datos

Se trabajará con tres datasets de clasificación de dígitos, ordenados de menor a mayor dificultad de transferencia desde MNIST:

Dataset	Dominio	Imágenes	Rol
MNIST	Dígitos escritos a mano, escala de grises	70 000	Entrenamiento
MNIST-M	Dígitos sobre fondos fotográficos a color	59 000	Evaluación / Target
SVHN	Dígitos en imágenes de casas reales	99 289	Evaluación

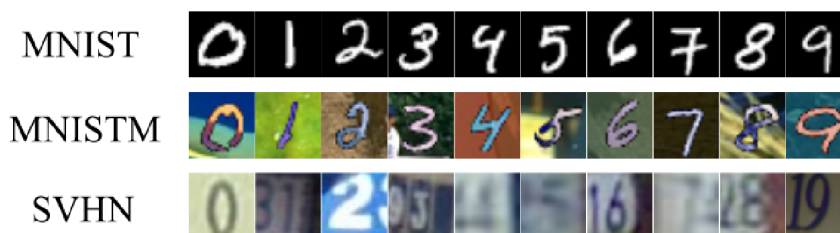


Figure 1: Ejemplos de dígitos de los tres dominios con los que se trabajará.

MNIST es el dominio fuente en todos los experimentos. **MNIST-M** (Ganin & Lempitsky, 2015) se construye reemplazando los fondos de MNIST por parches de imágenes fotográficas naturales, manteniendo la misma tarea de clasificación pero con una diferencia de dominio significativa. **SVHN** (*Street View House Numbers*) contiene imágenes reales capturadas en entornos urbanos, con variabilidad adicional de escala, rotación e iluminación.

Para las Partes 1 y 2, MNIST-M y SVHN se usarán exclusivamente en evaluación *cross-domain*, es decir, sin datos de estos dominios durante el entrenamiento. Para la Parte 3, las imágenes de MNIST-M (sin etiquetas) se incorporan al entrenamiento como dominio target para la adaptación adversarial.

Los datasets MNIST y SVHN se pueden descargar usando la librería `torchvision`. El dataset MNIST-M se puede descargar del siguiente link: https://drive.google.com/open?id=0B_tExHiYS-0vek1UZHfYT19KYjg

Objetivo

El objetivo del TP es implementar y comparar cuatro enfoques para la transferencia cross-domain en clasificación de dígitos:

- **Parte 1:** Prototypical Networks (ProtoNets).
- **Parte 2:** MAML y ProtoMAML.
- **Parte 3:** Domain-Adversarial Neural Network (DANN).

Etapas del Trabajo

4.1. Parte 1: Prototypical Networks

Las **Prototypical Networks** (ProtoNets, Snell et al. 2017) aprenden un espacio de embeddings donde clasificar una imagen de query consiste en encontrar el prototipo más cercano. Dado un episodio N -way K -shot, el prototipo de la clase n se obtiene promediando los embeddings del support set:

$$c_n = \frac{1}{K} \sum_{\substack{(x,y) \in \mathcal{D}^{\text{tr}} \\ y=n}} f_{\theta}(x)$$

La probabilidad de que una imagen de query x pertenezca a la clase n es:

$$p_{\theta}(y = n \mid x) = \frac{\exp(-\|f_{\theta}(x) - c_n\|^2)}{\sum_{n'} \exp(-\|f_{\theta}(x) - c_{n'}\|^2)}$$

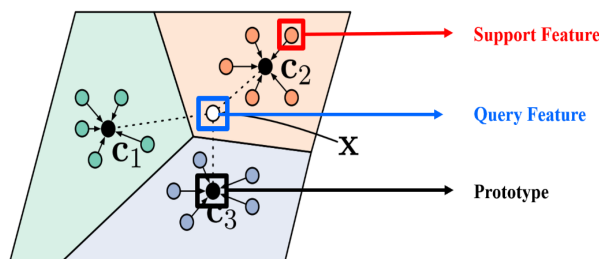


Figure 2: Prototypical Networks. En una tarea 3-way 5-shot, los prototipos de clase c_1 , c_2 , c_3 se computan como la media de los embeddings del support set (círculos de colores). Los prototipos definen regiones de decisión basadas en distancia euclídea. El ejemplo de query x se asigna a c_2 ya que sus features (círculo blanco) caen dentro de la región de esa clase.

4.1.1 Preparación de datos y episodios

- Usar MNIST como dominio de entrenamiento. Dividir los datos en train, validación y test. Recordar que el modelo se evaluará también en MNIST-M y SVHN en modo *few-shot*: en esos dominios solo se usarán imágenes de soporte para construir los prototipos, sin reentrenamiento.
- Implementar un **sampler episódico** que, en cada episodio, seleccione N clases al azar, K imágenes por clase para el support set, y Q imágenes distintas por clase para el query set.
- Usar episodios **5-way** durante el entrenamiento. Justificar la elección de arquitectura del encoder (se sugiere una ConvNet de 4 bloques o similar).

4.1.2 Entrenamiento y análisis in-domain

- Entrenar con Adam en episodios **5-way 5-shot** con $Q = 15$. Registrar las cuatro métricas de accuracy (support/query $\times$ train/val) y graficar la accuracy de query en validación a lo largo del entrenamiento.
- **Visualización del espacio latente (in-domain)**: proyectar con t-SNE o UMAP los embeddings del encoder para un conjunto de imágenes de MNIST al inicio, a mitad y al final del entrenamiento. Colorear los puntos por clase. ¿Se forman clusters separados? ¿Cómo evoluciona la estructura a medida que avanza el entrenamiento?

4.1.3 Evaluación cross-domain

- Evaluar el modelo entrenado en MNIST en episodios de test sobre **MNIST-M** y **SVHN** con $K \in \{1, 2, 4, 8, 16\}$ y $Q = 10$. Graficar la accuracy vs. K para los tres dominios en el mismo plot.
- **Visualización del espacio latente (cross-domain)**: proyectar en el mismo gráfico los embeddings de imágenes de MNIST y de MNIST-M (o SVHN) usando el encoder entrenado. Colorear por dominio y por clase. ¿Están los embeddings de los mismos dígitos cerca entre dominios? ¿Qué implica esto para la clasificación episódica cross-domain?

4.2. Parte 2: MAML y ProtoMAML

MAML (*Model-Agnostic Meta-Learning*, Finn et al. 2017) aprende parámetros iniciales θ que pueden adaptarse rápidamente a una tarea nueva con pocos pasos de gradient descent.

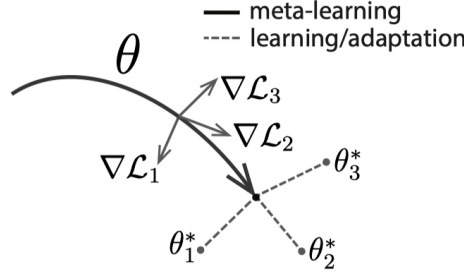


Figure 3: MAML. El algoritmo busca un vector de parámetros inicial θ que pueda adaptarse rápidamente, mediante gradientes de tarea, a vectores de parámetros óptimos para cada tarea específica ϕ_i^* .

El entrenamiento alterna entre dos loops:

- **Inner loop (adaptación):** para cada tarea $\mathcal{T}_i$, se realizan L pasos de SGD sobre el support set partiendo de θ :

$$\phi^\ell = \phi^{\ell-1} - \alpha \nabla_{\phi^{\ell-1}} \mathcal{L}(\phi^{\ell-1}, \mathcal{D}_i^{\text{tr}}), \quad \ell = 1, \dots, L$$

- **Outer loop (meta-optimización):** se actualiza θ con Adam minimizando la pérdida sobre el query set evaluada con los parámetros adaptados, diferenciando a través del inner loop:

$$J(\theta) = \mathbb{E}_{\mathcal{T}_i} [\mathcal{L}(\phi_i^L, \mathcal{D}_i^{\text{ts}})]$$

4.2.1 ProtoMAML: inicialización de la capa de salida

ProtoMAML (Triantafillou et al. 2020) usa los prototipos de ProtoNet para inicializar la capa de salida, proveyendo un punto de partida semántico para la adaptación. El resultado clave es que **la clasificación por distancia euclídea al prototipo es equivalente a una capa lineal**: dado un embedding de query $f_\theta(\mathbf{x}^*)$ y los prototipos de clase $\mathbf{v}_c$, podemos expandir la distancia euclídea al cuadrado:

$$-\|f_\theta(\mathbf{x}^*) - \mathbf{v}_c\|^2 = -f_\theta(\mathbf{x}^*)^\top f_\theta(\mathbf{x}^*) + 2\mathbf{v}_c^\top f_\theta(\mathbf{x}^*) - \mathbf{v}_c^\top \mathbf{v}_c$$

Al aplicar softmax sobre esta expresión para clasificar, el primer término $-f_\theta(\mathbf{x}^*)^\top f_\theta(\mathbf{x}^*)$ es constante respecto a la clase c y no afecta las probabilidades. Por lo tanto:

$$\text{softmax}_c(-\|f_\theta(\mathbf{x}^*) - \mathbf{v}_c\|^2) = \text{softmax}_c(2\mathbf{v}_c^\top f_\theta(\mathbf{x}^*) - \|\mathbf{v}_c\|^2)$$

Esto es exactamente la salida de una capa lineal $F(\mathbf{x}^*) = W f_\theta(\mathbf{x}^*) + b$ donde los pesos y biases de cada clase c se inicializan como:

$$W_c = 2\mathbf{v}_c^\top, \quad b_c = -\|\mathbf{v}_c\|^2$$

De este modo, antes de ejecutar cualquier paso del inner loop, la red ya produce predicciones equivalentes a un ProtoNet. Los pasos de adaptación refinan estos parámetros a partir de ese punto de partida, lo que requiere considerablemente menos pasos para converger en comparación con una inicialización aleatoria.

4.2.2 Implementación de MAML

- Implementar el inner loop usando `torch.autograd.grad` para no acumular gradientes en los parámetros del modelo. Diferenciar a través del inner loop en el outer loop (*second-order gradients*).
- Registrar seis métricas: accuracies pre y post-adaptación en support, y post-adaptación en query, en entrenamiento y validación.
- Entrenar en episodios **5-way 1-shot** con $Q = 15$ sobre MNIST.
- Graficar la accuracy post-adaptación en validación durante el entrenamiento.

4.2.3 Implementación de ProtoMAML

Implementar ProtoMAML: igual que MAML, pero inicializando los pesos W_c y biases b_c de la capa de clasificación a partir de los prototipos del support set, según la derivación de la sección anterior.

4.2.4 Comparación final: ProtoNet vs. MAML vs. ProtoMAML

- Para los tres modelos, evaluar en test episódico sobre los tres dominios (**MNIST**, **MNIST-M**, **SVHN**) con $K \in \{1, 2, 4, 8, 16\}$ y $Q = 10$. Graficar accuracy vs. K para cada dominio en un panel separado, mostrando los tres modelos en el mismo plot.
- **Visualización del espacio latente comparativa:** proyectar los embeddings del encoder para imágenes de MNIST, MNIST-M y SVHN en el mismo gráfico, usando el encoder de cada modelo. Colorear por dominio. ¿Qué modelo produce embeddings más cercanos entre dominios? ¿Se corresponde con la accuracy cross-domain?

4.3. Parte 3: Adaptación de Dominio Adversarial (DANN)

En las partes anteriores, el modelo se entrenó únicamente con MNIST y debió generalizarse a otros dominios sin ninguna información sobre ellos. La **Domain-Adversarial Neural Network** (DANN, Ganin et al. 2016) adopta un enfoque distinto: durante el entrenamiento se expone al modelo a imágenes *sin etiquetar* del dominio target (MNIST-M), forzando al encoder a producir representaciones que no permitan distinguir el dominio de origen.

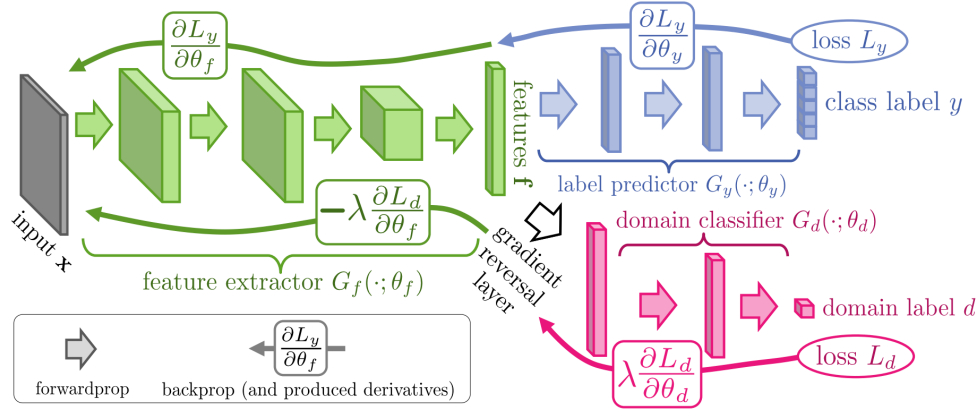


Figure 4: DANN

La arquitectura consta de tres componentes conectados al mismo encoder f_θ :

- **Clasificador de tarea g_y :** predice el dígito (10 clases) usando imágenes etiquetadas del source.
- **Clasificador de dominio g_d :** predice si una imagen proviene del source (MNIST) o del target (MNIST-M), conectado al encoder mediante una **Gradient Reversal Layer (GRL)**.

La GRL actúa como identidad en el *forward pass*, pero multiplica los gradientes por $-\lambda$ en el *backward pass*. Esto crea el juego adversarial: g_d intenta aprender a distinguir dominios, mientras que f_θ es penalizado si sus representaciones lo permiten. El objetivo combinado es:

$$\mathcal{L} = \mathcal{L}_{\text{cls}}(\text{MNIST}) - \lambda \cdot \mathcal{L}_{\text{dom}}(\text{MNIST} + \text{MNIST-M})$$

El coeficiente λ crece gradualmente durante el entrenamiento para evitar que el gradiente adversarial desestabilice el encoder en las etapas iniciales, cuando los features aún no son informativos:

$$\lambda_p = \frac{2}{1 + e^{-10p}} - 1, \quad p \in [0, 1]$$

4.3.1 Implementación

- **Gradient Reversal Layer:** implementar como módulo de PyTorch mediante `torch.autograd.Function`. En *forward*, identidad; en *backward*, multiplicar el gradiente por $-\lambda$.
- **Arquitectura:** encoder CNN compartido (misma arquitectura que en las partes anteriores para facilitar la comparación), clasificador de tarea (capas FC con ReLU) y clasificador de dominio (capas FC con ReLU).
- **Training loop:** en cada batch, combinar imágenes etiquetadas del source para $\mathcal{L}_{\text{cls}}$ con imágenes sin etiquetar de source y target para $\mathcal{L}_{\text{dom}}$. Actualizar λ a lo largo del entrenamiento según el esquema indicado.
- **Baseline:** entrenar un modelo idéntico *sin* GRL (solo $\mathcal{L}_{\text{cls}}$) para usar como referencia.

4.3.2 Análisis de resultados

- Graficar en el mismo plot la loss de clasificación, la loss de dominio y la accuracy en MNIST-M a lo largo del entrenamiento, para DANN y el baseline. ¿Cómo evolucionan en relación la una con la otra?
- Reportar en una tabla la accuracy final en source y target para: (a) baseline sin adaptación, (b) DANN.

4.3.3 Visualización del espacio latente: antes y después de la adaptación

Esta visualización es central para entender qué logra el entrenamiento adversarial. Proyectar con t-SNE o UMAP los embeddings del encoder para un conjunto fijo de imágenes de MNIST y MNIST-M en los siguientes momentos:

- **Antes del entrenamiento** (red preentrenada para MNIST solamente).
- **Finetuning** (sin GRL, solo $\mathcal{L}_{\text{cls}}$).
- **DANN** (con GRL y $\mathcal{L}_{\text{dom}}$).

Para cada visualización, generar **dos plots** con los mismos embeddings: uno coloreado por **dominio** (source vs. target) y otro coloreado por **clase** (dígito 0–9). Analizar.

Informe y Grupos de Trabajo

Deberán presentar un informe de **no más de 10 páginas** exponiendo las decisiones de diseño, implementación y resultados de cada parte. El trabajo es **grupal, en grupos de dos personas**.

El informe debe contener al menos las siguientes secciones:

1. **Introducción:** motivación, descripción del problema y estructura del trabajo.
2. **Método:** descripción de cada técnica implementada, incluyendo funciones de pérdida, hiperparámetros elegidos y justificación de las decisiones tomadas.
3. **Resultados:** tablas, curvas y visualizaciones obtenidas en cada parte. Todos los gráficos deben incluir ejes etiquetados, leyenda y título.
4. **Conclusiones:** análisis comparativo de los métodos y reflexiones sobre sus supuestos y limitaciones.

Formatos y Fechas de Entrega

El informe y el código fuente se entregarán juntos mediante el Campus Virtual en un archivo `.zip` denominado `apellido1_apellido2_tp2.zip`.

La estructura del directorio debe ser la siguiente:

```
apellido1_apellido2_tp2.zip/
  imgs/
  apellido1_apellido2_tp2_informe.pdf
  tp2.ipynb          (o el formato de código utilizado)
```

Pueden entregar el código en el formato que hayan utilizado (notebook `.ipynb`, scripts `.py`, etc.). Lo importante es que el código sea reproducible: incluir instrucciones claras para ejecutarlo.